

Notes on Thompson Sampling

1 The Multi-Armed Bandit Problem

We have d arms (in the ads example, $d = 10$ ads). At each round $n = 1, \dots, N$ we select one arm $i \in \{1, \dots, d\}$ and observe a reward $r_i(n) \in \{0, 1\}$ (click / no click), drawn from an unknown Bernoulli distribution with parameter θ_i specific to that arm:

$$r_i(n) \sim \text{Bernoulli}(\theta_i).$$

The θ_i (true click-through rates) are unknown and must be estimated online. The goal is to maximize cumulative reward, i.e. balance **exploration** (trying arms we are unsure about) against **exploitation** (playing the arm that currently looks best).

2 Bayesian Idea

Thompson Sampling treats each θ_i as a random variable and maintains a posterior belief over it, updated after every observation. At each round it *samples* a value from each arm's posterior and picks the arm with the highest sampled value — i.e. it acts greedily with respect to a random draw from what it currently believes, rather than a fixed statistic (like UCB's upper confidence bound).

Prior

Since rewards are Bernoulli, the conjugate prior for θ_i is the Beta distribution. We start with an uninformative prior for every arm:

$$\theta_i \sim \text{Beta}(\alpha_i, \beta_i), \quad \alpha_i = 1, \beta_i = 1 \quad (\text{i.e. Uniform}(0, 1)).$$

Posterior update (conjugacy)

Let

$$N_i^1(n) = \text{number of times arm } i \text{ received reward 1 up to round } n,$$

$$N_i^0(n) = \text{number of times arm } i \text{ received reward 0 up to round } n.$$

Because Beta is conjugate to Bernoulli, the posterior after observing these counts is again Beta:

$$\theta_i \mid N_i^1(n), N_i^0(n) \sim \text{Beta}(N_i^1(n) + 1, N_i^0(n) + 1).$$

This is exactly `numbers_of_rewards_1` and `numbers_of_rewards_0` in the implementation, and it is why the sampling step is `random.betavariate(N1[i] + 1, N0[i] + 1)`.

Algorithm 1 Thompson Sampling for Bernoulli bandits

```
1:  $N_i^1 \leftarrow 0, N_i^0 \leftarrow 0$  for all arms  $i = 1, \dots, d$ 
2: for  $n = 1$  to  $N$  do
3:   for  $i = 1$  to  $d$  do
4:     draw  $\hat{\theta}_i \sim \text{Beta}(N_i^1 + 1, N_i^0 + 1)$ 
5:   end for
6:   select arm  $i^* = \arg \max_i \hat{\theta}_i$ 
7:   observe reward  $r \in \{0, 1\}$  for arm  $i^*$ 
8:   if  $r = 1$  then
9:      $N_{i^*}^1 \leftarrow N_{i^*}^1 + 1$ 
10:  else
11:     $N_{i^*}^0 \leftarrow N_{i^*}^0 + 1$ 
12:  end if
13: end for
```

3 Algorithm

4 Why It Works: Exploration vs. Exploitation

- Early on, with few observations, $\text{Beta}(1, 1)$ or $\text{Beta}(\text{small}, \text{small})$ is wide/flat, so sampled $\hat{\theta}_i$ values vary a lot round to round $\Rightarrow$ arms get explored roughly uniformly.
- As N_i^1, N_i^0 grow for a given arm, the Beta posterior concentrates around the empirical rate $\frac{N_i^1}{N_i^1 + N_i^0}$, so sampled values become consistent and predictable $\Rightarrow$ the algorithm exploits the arm if it truly is good.
- Arms that look bad get sampled less, so their posteriors stay wide for longer, keeping a nonzero (but shrinking) chance they still get explored, in case early evidence was misleading.

This randomized-but-principled trade-off is why Thompson Sampling tends to outperform simpler heuristics and even Upper Confidence Bound (UCB) in practice, especially with delayed/batched feedback.

5 Thompson Sampling vs. UCB

	UCB	Thompson Sampling
Nature	Deterministic	Probabilistic (stochastic)
Selection rule	$\arg \max_i \left(\bar{r}_i + \sqrt{\frac{3}{2} \frac{\log n}{n_i}} \right)$	$\arg \max_i \hat{\theta}_i, \hat{\theta}_i \sim \text{Beta}(N_i^1 + 1, N_i^0 + 1)$
Feedback timing	Needs update every round	Can tolerate delayed/batched updates
Empirical performance	Good	Often better in practice

6 When to Use Which

- **UCB** is deterministic and easy to reason about: given the same history, it always makes the same choice. It needs the full cumulative history recomputed every round, and it forces every arm to be tried once up front (an infinite/very large upper bound is assigned to unseen arms,

seen in code as `upper_bound = 1e400`). It has a tunable exploration constant (the $\sqrt{3/2 \cdot \dots}$ term) that controls how aggressively it explores. Good fit for stationary problems where reward feedback arrives immediately every round and you want a reproducible, auditable selection rule.

- **Thompson Sampling** is probabilistic: it only needs the latest success/failure counts (not the full history) to draw a sample, so it updates cheaply and incrementally. There is no tunable exploration parameter — the Beta(1,1) prior is naturally wide for unseen arms, so cold start is handled for free. It also tends to tolerate delayed or batched reward feedback better, since stale posteriors just stay wide instead of stalling the algorithm. This is why Thompson Sampling is generally preferred in production settings such as ad serving, online A/B testing, and recommendation systems, and why it converges faster / earns a higher `total_reward` than UCB on this dataset in practice.

7 Correspondence to the Notebook Code

Math	Code (<code>thompson_sampling.ipynb</code>)
d	<code>d = 10</code> (number of ads)
N	<code>N = 10000</code> (number of rounds/users)
$N_i^1(n)$	<code>numbers_of_rewards_1[i]</code>
$N_i^0(n)$	<code>numbers_of_rewards_0[i]</code>
$\hat{\theta}_i \sim \text{Beta}(N_i^1 + 1, N_i^0 + 1)$	<code>random.betavariate(N1[i]+1, N0[i]+1)</code>
$i^* = \arg \max_i \hat{\theta}_i$	<code>if random.beta > max_random: ad = i</code>
cumulative reward	<code>total_reward</code>